

Assignment 2- Case Study (8995-G level Task)

Stage 5 Lab Activities

Refactoring SmartCare into a Layered Architecture

HUMAN ANALYSIS -> REFACTOR -> AI REVIEW -> VERIFY | 1 hour

A - Inspect SmartCare v0.4

Identify domain, workflow, data-access and presentation responsibilities currently mixed together.

B - Propose Architecture

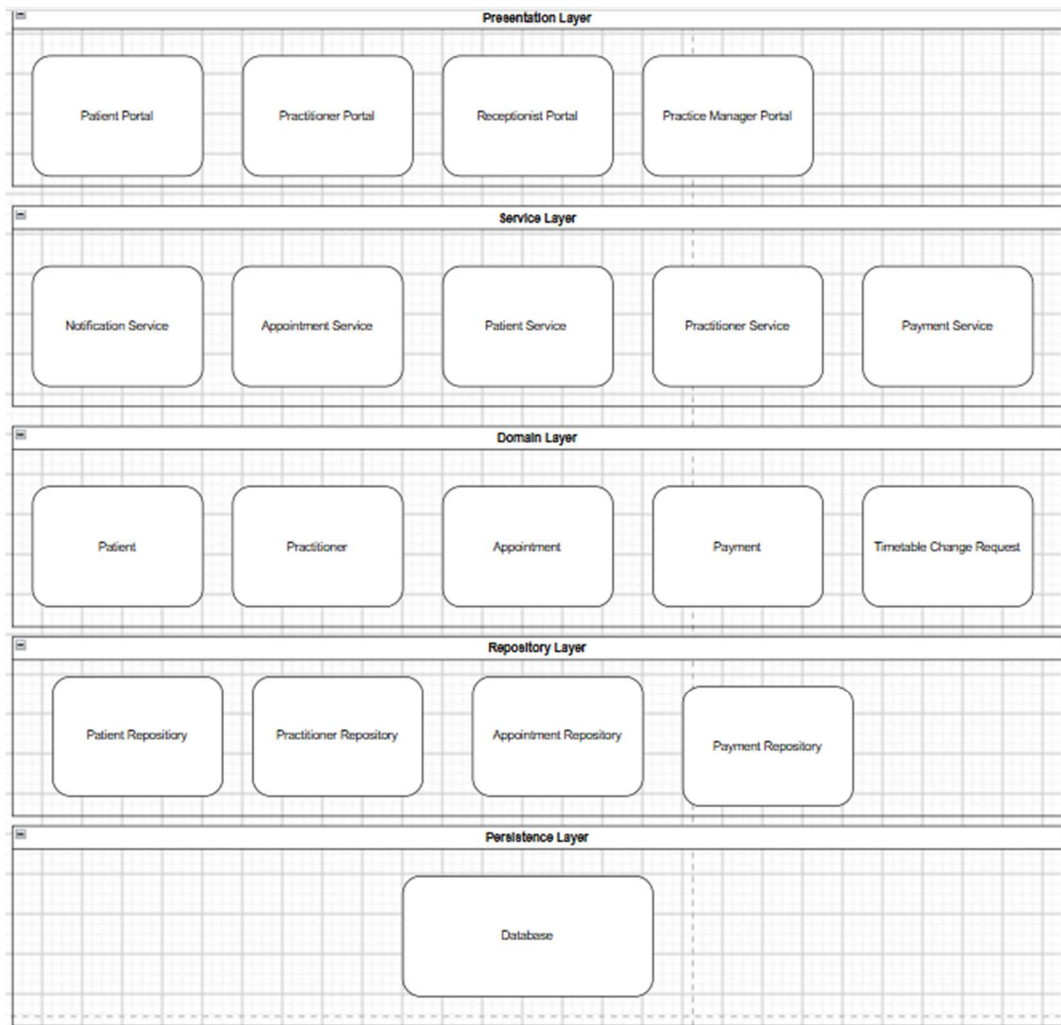
Draw Presentation -> Service -> Domain, with Service using a Repository abstraction and Persistence implementing it.

Problem	Evidence	Impact	Refactoring
Syntax errors	The previous code had multiple syntax errors and could use validation	The software produces incorrect results or behaves unpredictably because the logic does not match the intended workflow.	Correct all syntax mistakes, run code validation tools, and ensure the program compiles before adding new features.
Logical errors	The code did not operate using typical logic code	The software produces incorrect results or behaves unpredictably because the logic does not match the intended workflow.	Rewrite the logic to follow correct control flow, add conditional checks, and test each function to confirm expected behaviour.
The database is already in the database.	The database will not be uploaded properly because a database will already exist. This can be shown with the new statement that will be added: IF NOT EXISTS.	The system may crash or fail to initialise because it attempts to recreate an existing database, causing duplication or errors.	Add IF NOT EXISTS to database creation statements and implement checks to avoid recreating existing structures.
Indentation	In order to make it clearer to understand, indentation should be properly observed; this can be reviewed with regard to past code.	Poor readability makes the code harder to maintain, debug, or extend, increasing the chance of future mistakes.	Apply consistent indentation and formatting standards across all files to improve clarity and maintainability.
Commenting	Commenting was not shown in previous code, making it hard to understand the direction	Without comments, it becomes difficult to understand the purpose of functions or the	Add clear comments about function roles to guide future maintenance and

	the software was meant to take.	intended workflow, slowing down development.	collaboration.
--	---------------------------------	--	----------------

C - Create Package Structure

Create domain/, services/, repositories/, persistence/ and presentation/ or a justified equivalent.



D - Introduce AppointmentService

Move workflow coordination into a focused service without stealing Appointment domain behaviour.

E - Repository Abstraction

Define a small AppointmentRepository contract using only current use-case needs.

```

import sqlite3

connection = sqlite3.connect("Smartcare.db")

mutator = connection.cursor()

#Create patient's medical record table
mutator.execute('''CREATE TABLE IF NOT EXISTS patientRecord(
patientID int PRIMARY KEY,
mrn int NOT NULL,
medicalHistoryID int,
diagnosisID int,
firstName varchar (70) NOT NULL,
lastName varchar (700) NOT NULL,
phoneNumber varchar (15) NOT NULL,
currentMedication varchar(30),
FOREIGN KEY (medicalHistoryID) REFERENCES medicalHistory(medicalHistoryID),
FOREIGN KEY (diagnosisID) REFERENCES diagnosis(diagnosisID)
)''')

#Create patient's medical history table
mutator.execute('''CREATE TABLE IF NOT EXISTS medicalHistory(
medicalHistoryID int PRIMARY KEY,
diagnosisID int,
allergies varchar(40) NOT NULL,
hereditaryConditions varchar(40),
medicalConcern varchar(200) NOT NULL,
FOREIGN KEY (diagnosisID) REFERENCES diagnosis(diagnosisID)
)''')

#Create patient's diagnosis table
mutator.execute('''CREATE TABLE IF NOT EXISTS diagnosis(
diagnosisID int PRIMARY KEY,
dateTime datetime NOT NULL,
generalPractitioner varchar(770) NOT NULL,
anatomicalSite varchar(50) NOT NULL,
diagnosisName varchar(50) NOT NULL,
clinicalStage varchar(9),
onsetSpeed varchar(20),
conditionStatus varchar(30) NOT NULL,
laterizedLesion varchar(30) NOT NULL,
prescribedMedication varchar(30)
)''')

```

F - AI Architecture Review

Ask AI to review dependency direction, misplaced responsibilities and unnecessary complexity; request simplest justified improvements.

COPILLOT AI PROMPT:

1. Dependency Direction Problems

A clean schema flows **one way: Patient → MedicalHistory → Diagnosis → Practitioner/TestResults**

But your schema currently has **cycles**:

- patientRecord → medicalHistory

- medicalHistory → diagnosis
- patientRecord → diagnosis

This creates a loop: **patientRecord** → **medicalHistory** → **diagnosis** → **patientRecord**

SQLite will allow it, but it makes inserts, updates, and deletes harder.

✓ Simplest fix

Remove **diagnosisID** from patientRecord. A patient doesn't "own" a diagnosis — diagnoses belong to medical history or encounters.

Why this is justified:

- A patient can have multiple diagnoses over time.
- A diagnosis should not be a direct child of patientRecord.
- It breaks the cycle cleanly.

⚙️ 2. Misplaced Responsibilities

Some tables are doing jobs they shouldn't.

✗ patientRecord is overloaded

It currently mixes:

- identity
- contact info
- medical history linkage
- diagnosis linkage
- medication

This is too much for one table.

✓ Simplest fix

Split into two responsibilities:

patient

- patientID
- firstName
- lastName
- phoneNumber

patientRecord

- patientID (FK)
- mrn
- medicalHistoryID
- currentMedication

Why this is justified:

- Identity and clinical data should not live together.
- Makes updates safer and reduces accidental overwrites.
- Matches real-world EMR design.

✗ medicalHistory contains diagnosisID

Medical history is long-term. Diagnosis is episodic.

✓ Simplest fix

Remove diagnosisID from medicalHistory.

Instead, let **diagnosis reference medicalHistory**.

Why this is justified:

- A diagnosis is an event in time.
- Medical history is a stable record.
- This aligns with clinical reality.

✗ practitionerRecord contains diagnosisID

A practitioner does not “own” a diagnosis. A diagnosis is authored by a practitioner.

✓ Simplest fix

Move practitionerID into diagnosis.

Why this is justified:

- A diagnosis is created by a practitioner.
- A practitioner may create many diagnoses.
- This removes unnecessary coupling.

🔧 3. Unnecessary Complexity

SQLite is simple. Your schema is more complex than needed for a small system.

✗ **bookingSession has appointmentDuration as datetime**

Duration is not a timestamp.

✓ **Simplest fix**

Change to **integer minutes**.

✗ **medicalProvider is separate from practitionerRecord**

Unless SmartCare distinguishes “provider” vs “practitioner”, this is redundant.

✓ **Simplest fix**

Merge medicalProvider into practitionerRecord.

✗ **Too many foreign keys for a small SQLite app**

Foreign keys are good, but SQLite’s FK enforcement is limited.

✓ **Simplest fix**

Keep only the **essential** foreign keys:

- patientRecord → patient
- medicalHistory → patientRecord
- diagnosis → medicalHistory
- diagnosis → practitioner
- testResult → diagnosis

Everything else can be enforced at the application layer.

🧱 **4. Cleanest Minimal Schema (Improved)**

patient

patientRecord

medicalHistory

diagnosis

practitioner

testResult

bookingSession

No cycles. No overloaded tables. No redundant relationships.

G - Refactor and Verify

Apply only justified changes and confirm required behaviour remains unchanged.

H - Reflection

Document one AI suggestion accepted, one modified and one rejected/deferred.

Suggested AI prompt

Act as a software architecture reviewer. Review this small SmartCare Python system against separation of concerns, cohesion, coupling and introductory SOLID principles. Identify concrete layer violations and dependency risks. Prefer the simplest refactoring that solves an observed problem. Do not introduce frameworks, microservices or patterns unless current requirements justify them.